Hiking-Tour-Assistant

Design Report
23.3.2026

Ville Elamo
Kaarle Sarnes
Max Ihalempiä

1. Introduction
    1.1. Purpose
    1.2. Scope
    1.3. Code repository
    1.4. Definitions, acronyms, and abbreviation
2. Overall description
3. Data design
    3.1. Smartwatch
        3.1.1. Data Description
        3.1.2. Data Dictionary
    3.2. Raspberry Pi
        3.2.1. Data Description
4. System Architecture
    4.1 Smartwatch
        4.1.1 Smartwatch architecture
        2.1.2 Components/modules
    4.2 Data processing application
        4.2.1 Data processing application architecture
        4.2.2. Components
5. User interface design
    5.1. Smartwatch
        5.1.1. Overview of user interface
        5.1.2. Screen pictures
    5.2. Raspberry Pi
        5.2.1. Overview of user interface
    5.3 WebUI
        5.3.1. Overview of user interface
        5.3.3. Using the WebUI
6. ATAM study

1. Introduction

1.1. Purpose

The purpose of this document is to provide a comprehensive review on the software of the Hiking Tour Assistant system. This specification is intended for use by software developers and the ELEC-E8408 Embedded Systems Development course staff.

1.2. Scope

This software package is part of a dual component system composed of a LilyGoT-Watch 2020 V2 smart watch and external Raspberry Pi computer.

The smartwatch will record real-time hiking data, including step count, traveled distance and session duration. After a session has ended, the data will be sent using Bluetooth to the Raspberry Pi. The Raspberry Pi provides a browser-based local interface for a user to review the latest hiking session data.

This system is intended for personal data logging. The primary purpose is to provide a hiker a self-hosted, easily modifiable alternative to commercial devices.

1.3. Code repository

The codebase for this project is hosted onGitHub: https://github.com/xdvii7/Hiking-Tour-Assistant

1.4. Definitions, acronyms, and abbreviation

Smartwatch	LILYGO T-WATCH-2020 V2
RPi	Raspberry Pi single-board computer
SRS	Software Requirement Specification
WebUI	Web User Interface

2. Overall description

The system consists of two individual software packages, one is the smartwatch firmware and one is the application for Raspberry Pi. The main functionalities of the dual system are:

- Recording, and saving hike a hike session with a smartwatch and displaying the hike statistics to the user during recording
- Calculating burnt calories on the Raspberry Pi
- Storing hike session data on an a database on the Raspberry Pi
- Viewing hike session data on a WebUI

3. Data design

3.1. Smartwatch

3.1.1. Data Description

The hike data is stored in variables during the hiking session. The variables are used to store and display the data during and after a session. After the session, the variables are saved to different text files using the LITTLEFS library.

The data is sent automatically to the Raspberry Pi when a Bluetooth connection has been formed between the smartwatch and the Raspberry Pi. The variables in the smartwatch are overwritten when a new hiking session begins.

3.1.2. Data Dictionary

File	Variable	Data type	Description	Storage format description
distance.txt	-	-	Calculated from the steps (1 step = 0.8 m)	float (km)
id.txt	sessionId	uint32_t	Unique id of the session on the smartwatch	integer
time.txt	totalSeconds	uint32_t	Total duration of the session in seconds	integer
steps.txt	steps	uint32_t	read from the BMA423 module	integer

3.2. Raspberry Pi

3.2.1. Data description

The received hike session data is handled by the Raspberry Pi, which inserts calculated burnt calories into the session data. This session data is saved then into a database file called sessions.db via SQLite. The SQLite database structure and format are showcased below in the table. Data can be fetched and removed from the database. The removal is

implemented via the WebUI button that sends SQL DELETE-command into a specific hike session in the database.

```
DB_SESSION_TABLE = {
    "name": "sessions",
    "cols": [
        "session_id integer PRIMARY KEY",
        "km integer",
        "steps integer",
        "burnt_kcal integer",
        "duration integer"
    ]
}
```

Figure 1: DB_SESSION_TABLE and its variables

File	Variable	Data type	Description	Storage format
sessions.db	DB_SESSION_TABLE	SQLite database file .db	The table contains the hike session data stored in the Raspberry Pi (variables below)	All are SQLite integer
sessions.db	session_id	Integer	Primary key, represents a hike session	SQLite integer
sessions.db	km	Integer	Travelled kilometers in a hike session (from steps)	SQLite integer
sessions.db	steps	Integer	Steps in a hike session	SQLite integer
sessions.db	burnt_kcal	Integer	Burnt calories from a hike session	SQLite integer
sessions.db	time	Integer	Seconds spent in a hike session	SQLite integer

4. System Architecture

4.1. Smartwatch

The Smartwatch main program is located on the main.cpp file. It is structured with two main functions: setup() and loop() in arduino style. The setup function includes initialization of the watch. The loop includes a state machine for continuous execution.

4.1.1. Smartwatch architecture

The smartwatch executes as a finite state machine. The state transitions are triggered by the user pressing the button or internal logic.

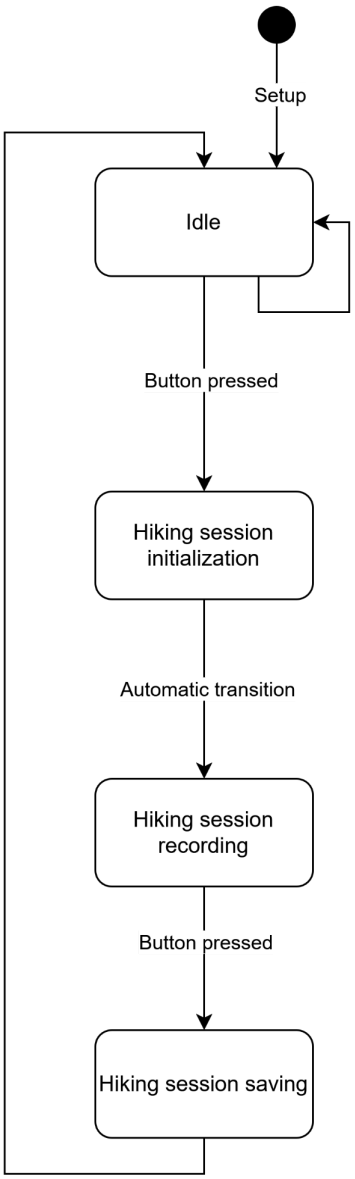

Figure 2: The UML State machine diagram.

Idle
The default state where the watch will enter after it's turned on and the initial setup is done. The watch will display instructions to initialize the hiking session by the button. The watch tries to look for bluetooth connection and begins the bluetooth synchronization sequence if connection is found. The watch will check if the button press flag is active and if so, it enters the session initialization state.

Session initialization
The session initialization state displays a starting hike message. Session start time is recorded. Watch will enter the session recording state automatically.

Session recording
The session records the state variables, that are steps, duration and distance, and displays them, until the button is pressed again.

Session saving
When the button has been pressed, the hiking session stops and the hike data is saved. Once the saving is done, the smartwatch returns to the *idle* state.

2.1.2. Components/modules

Data storage (LittleFS): Filesystem used to store data on the ESP32's internal flash memory.

watch (TTGOClass): Manages access to watch hardware such as display, BMA sensor and power management.

Bluetooth communication (SerialBT): Manages bluetooth communication with Raspberry Pi.

sensor (BMA): Represents Bosch BMA423 accelerometer, which is used to count steps.

tft (TFT_eSPI): Represent the LCD screen of the watch.

4.2. Data processing application

4.2.1. Data processing application architecture

Raspberry Pi has a data processing application that functions as a data processing, storing and presenting system. There are two concurrent components, the receiver and web server, which share the same database for storing sessions.

The classes of receiver application are presented in the Class diagram.

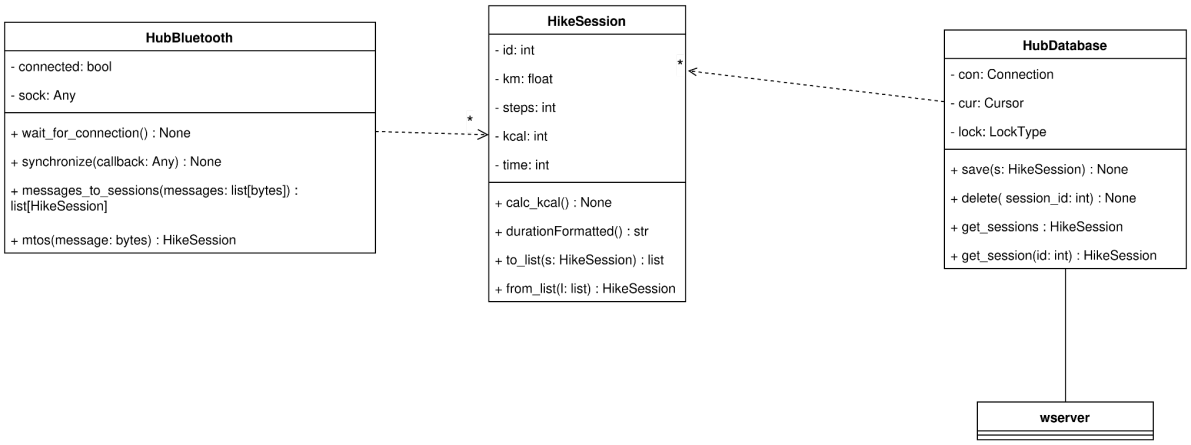

Figure 3: Class diagram of receiver system

Sequence diagram (Fig. 4) displays the logic of the receiver component. Receiver app is first launched. It creates HubBluetooth, which handles the communication with smartwatch, and HubDatabase, which handles the database operations with SQLite3 library.

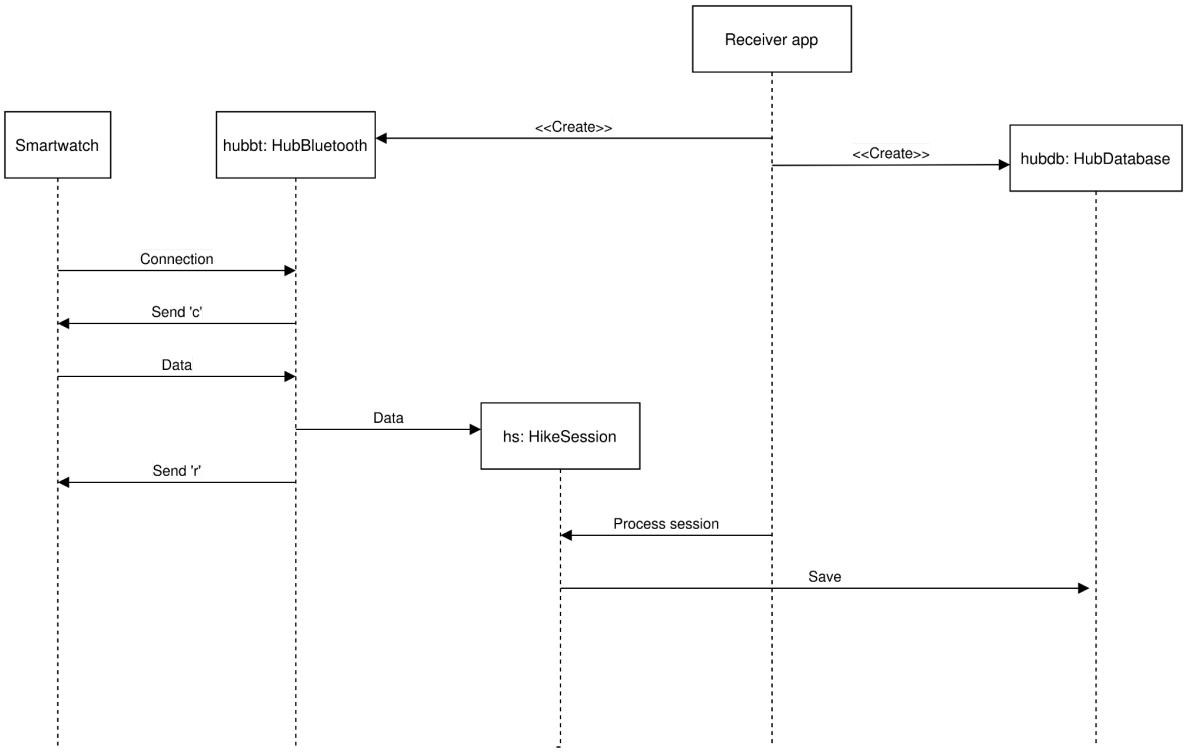

Figure 4: Sequence diagram of receiver system

Sequence diagram (Fig. 5) displays the logic of the wserver component. Wserver app is first launched. It creates the HubDatabase object, which handles the database connection.

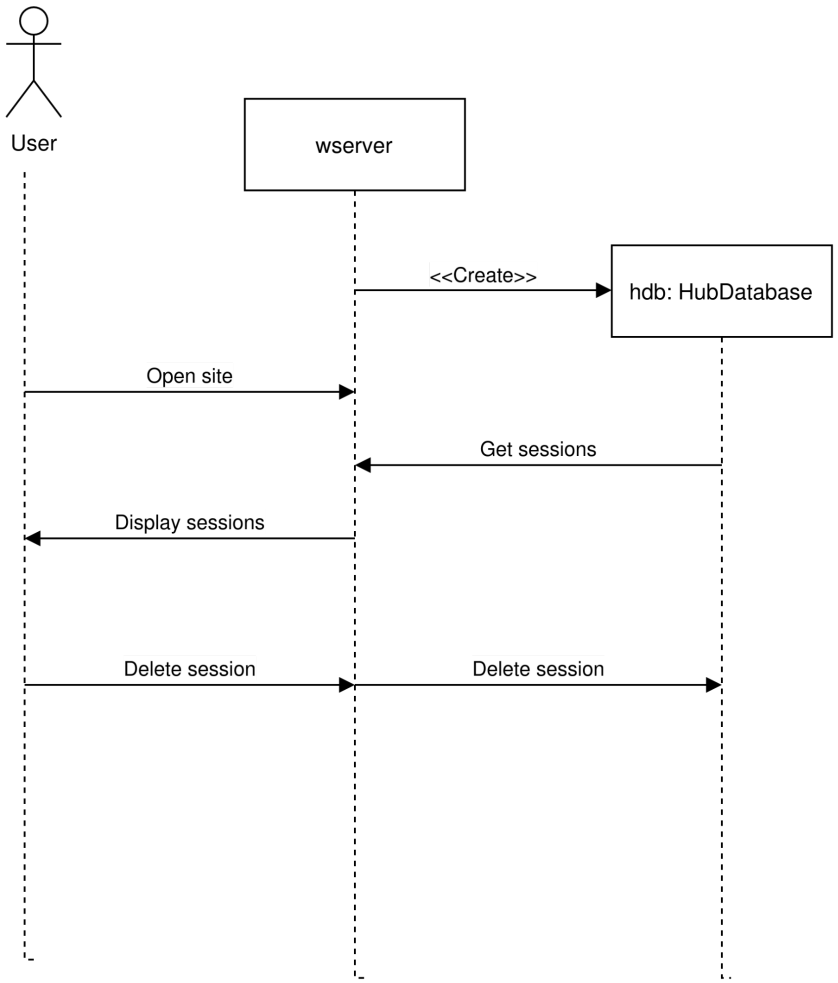

Figure 5: Sequence diagram of web server application

4.2.2. Components

HubBluetooth (Bluetooth): a class that handles the connection and synchronization between the smartwatch and the Raspberry Pi.

Receiver (sqlite3): initializes the Bluetooth connections and tries to establish it between the smartwatch and the Raspberry Pi.

HubDatabase (sqlite3): a class interface for managing the hiking session data in the SQLite database. Takes care of different operations related to the hiking session data.

HikeSession: a class to represent a hiking session. Includes all the relevant data related to a hiking session, such as steps, duration and distance.

Wserver (Flask): Establishes the web server using Flask. Defines handlers and routes for HTTP requests for the server to handle.

5. User interface design

5.1. Smartwatch

5.1.1. Overview of user interface

The smartwatch is intended to be easy to use. The user controls the smartwatch by using only the button.

The smartwatch is turned on by holding the button. The smartwatch goes to the starting screen (Fig. 6). When the button is pressed, the next hiking session starts (Fig. 7). During the sessions, the steps, distance and duration are visible to the user. A new press ends the current session and sends the data to the Raspberry Pi when the Bluetooth connection has been formed (Fig 8). After this, the smartwatch returns to the starting screen.

Data synchronization is autonomous between the smartwatch and the Raspberry Pi, given a Bluetooth connection.

5.1.2. Screen pictures

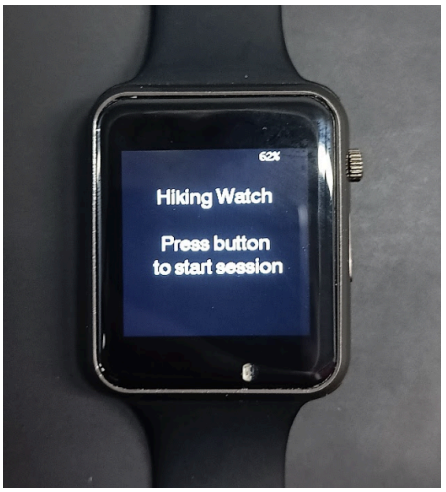

Figure 6: the starting screen

Figure 7: the session screen

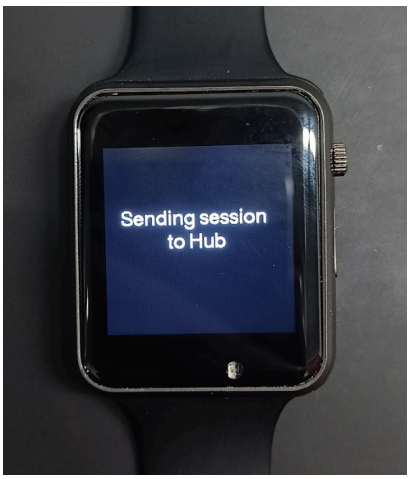

Figure 8: sending data to Raspberry Pi.

5.2. Raspberry Pi

5.2.1. Overview of user interface

The Raspberry Pi is used by writing commands into its terminal.

5.3 WebUI

5.3.1. Overview of user interface

The user can view and delete previous sessions' information on the WebUI. Additionally, the user can see the combined hike statistics over the individual hike statistics.

5.3.2. Using the WebUI

The user can automatically see the latest and all previous sessions and their data in the web browser. If the user wants to delete a session, they can press the red button (Delete). If a new session arrives at the Raspberry Pi, the page must be refreshed to view it.

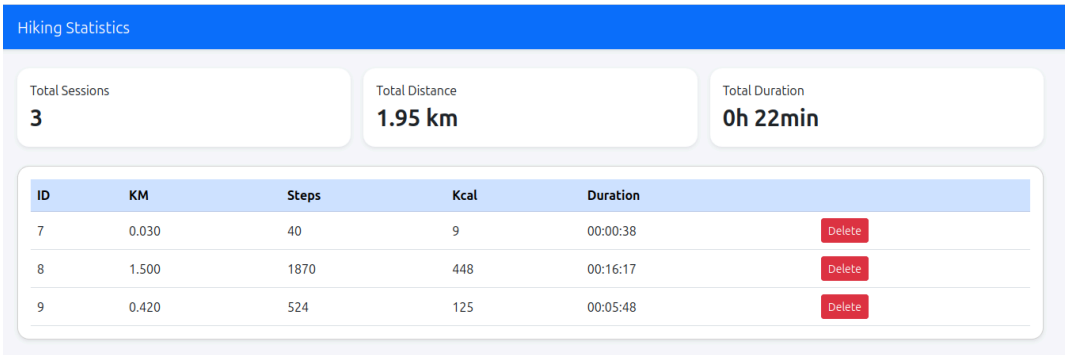

Figure 9: the WebUI

5. Libraries and Tools

5.1. Smartwatch

The software of the smartwatch is a PlatformIo project. The libraries included are TTGO TWatch library and LittleFS_esp32.

5.2. Raspberry Pi

Flask for web applications, sqlite3 for data storage and Bluetooth (pyBluez-module) for connection with the watch.

5.3. WebUI

The WebUI utilizes the following tools and libraries: Bootstrap and CSS.

6. ATAM study

Quality Attribute	Attribute refinement	Perspective	Scenario	Votes (total of 30)
Monitorability	Session status	User	Is able to view the session variables during a session.	4
	Session data	User	Is able to view earlier sessions on the Raspberry Pi on a web browser.	4
Performance	Session initialization time	User	The initialization of a session takes less than 2 seconds.	2
	Session performance	User	Can view the session statistics during a session in real time. The screen updates at least at the rate of 100 ms.	3
	Upload performance	User	The uploading of a session to the Raspberry Pi takes less than a minute.	2
Reliability	Restarting	User	In case of the smartwatch freezing, the smartwatch can always be reset by turning the device off and off.	4
Automation	Uploading	User	The data is automatically uploaded on the Raspberry Pi after a session when they are close enough of each other.	1

Flexibility	How easy it is to add features	Software developer	All the software is easily available and modifiable.	2
	Device compatibility	Smartwatch developer	The same software can be used on different versions of the smartwatch.	2
Scalability	Software availability	Smartwatch developer	Because all the software is freely available, it can be uploaded to as many smartwatches as needed.	2
Maintainability	Documentation level of the interfaces	Software devel-oper	Ability to get overview of the interfaces between the subsys-tems.	1
Transparency	Quality and availability of documentation	software developer	All the relevant code is freely available and well documented.	1
Security	Area of access	User	A connection to the smartwatch can only be set in the immediate vicinity of the smartwatch.	2

Architectural decision	Type	Consequences
Most of the data is stored on the Raspberry Pi.	non-risk	Even if the smartwatch breaks, all the old data is safely stored on the Raspberry Pi.
The webUI is on a local web Browser.	non-risk	Not needing an internet Connection simplifies the required architecture and reduces the risk of the

		sensitive data getting stolen.
Current architecture does not display many of the important smartwatch features such as Bluetooth, wifi or battery data.	risk	The user has no idea of the state of the smartwatch other than whether it is on or off.
The Bluetooth is constantly on on the smartwatch.	risk	The Bluetooth being constantly on simplifies the software and the interaction of the user but creates the possibility of malicious actors getting access into the smartwatch.

Tradeoff description	Affected quality attributes
Having a removable battery	Fixability (battery can be replaced), Durability (the smartwatch is not watertight)
Bluetooth constantly on	Usability, Security
The smartwatch saves can only save one session	Simplicity, flexibility, security
Watch is controlled using only the button	Usability, simplicity, flexibility